

Labwork 7: Reduction

Ngo Quang Truong 2540106

3 Oct 2026

1 Introduction

This report presents a grayscale stretch implemented on GPU by combining the Map and Reduction patterns.

2 Method

The grayscale stretch has three steps:

1. **Map**: convert each RGB pixel to gray, $g = (R + G + B)/3$ (2D kernel, 16×16 block).
2. **Reduce**: find the minimum and maximum intensity of the gray image.
3. **Map**: linearly stretch each pixel from $[min, max]$ to $[0, 255]$:

$$g' = \frac{g - min}{max - min} \times 255 \quad (1)$$

Test image: a 3840×2160 RGB image.

3 Reduction

The gray image is flattened to a 1D array and split into blocks of 256 threads.

- Each thread copies one pixel into two shared-memory arrays (one for min, one for max).
- The block halves the stride at each step ($s = 128, 64, \dots, 1$); thread $t < s$ keeps the min/max of t and $t + s$. This avoids branch divergence and bank conflicts.
- Thread 0 writes the block result to global memory.
- The few thousand block results are copied back and finished on the CPU with `min()` / `max()`.

The GPU result was checked against `numpy.min` and `numpy.max`.

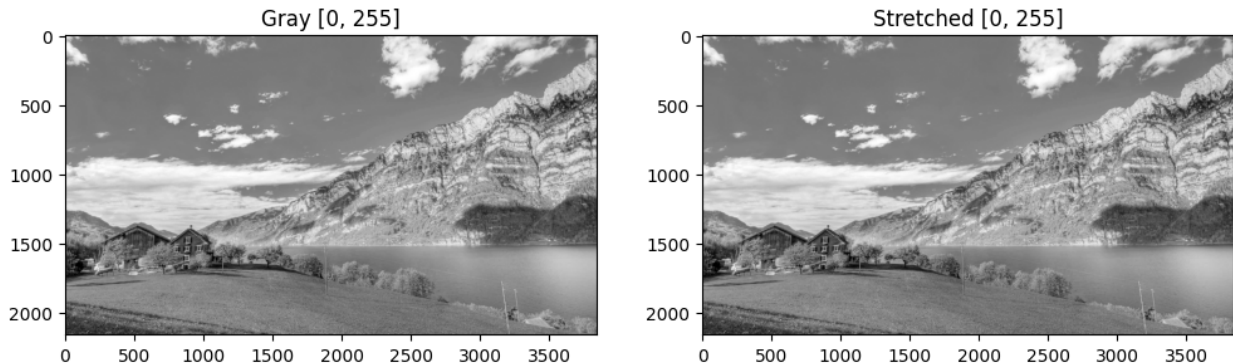


Figure 1: Gray image (left) vs. stretched output